

CICLO DE VIDA DEL DESARROLLO DE SOFTWARE (SDLC)

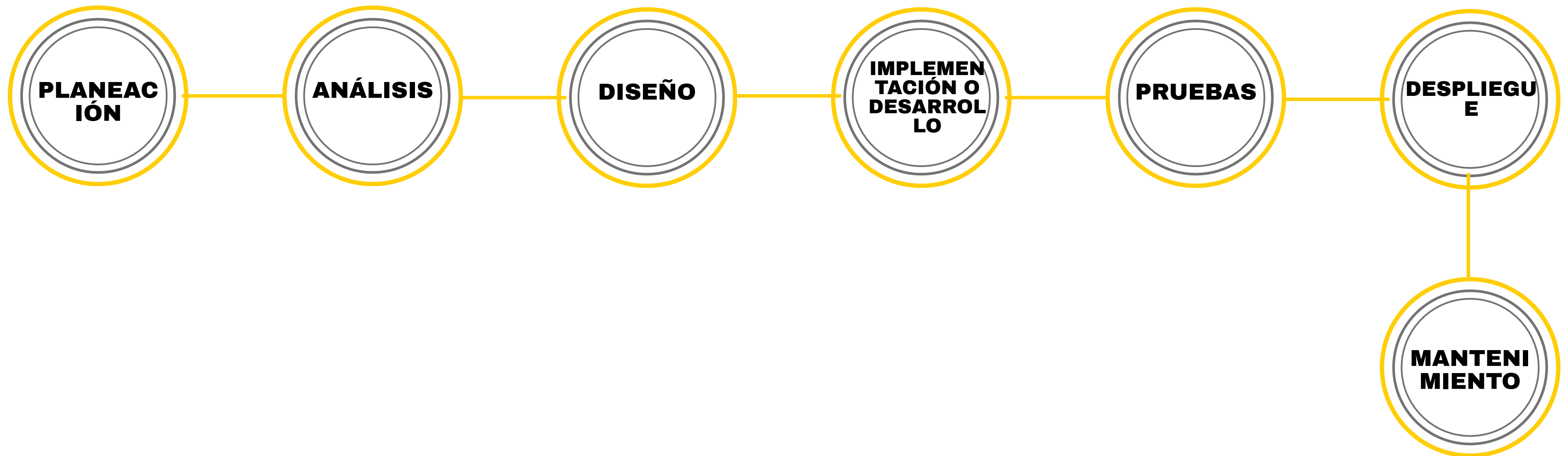
Preguntas y Respuestas Esenciales para comprender el
proceso de desarrollo de software de calidad.

¿Qué significa SDLC?

SDLC significa Software Development Life Cycle (Ciclo de Vida del Desarrollo de Software). Es un marco metodológico que define las etapas necesarias para desarrollar software de alta calidad. Su objetivo principal es organizar y guiar todo el proceso de desarrollo, asegurando que el producto final sea eficiente, cumpla con los requisitos del usuario y se entregue dentro del tiempo y presupuesto establecidos.



Menciona las 7 fases principales del desarrollo de software.





¿Por qué es importante definir bien el alcance de un proyecto?

Definir claramente el alcance establece con precisión qué funcionalidades tendrá el sistema y cuáles quedan fuera. Esto proporciona una guía clara para todo el equipo de desarrollo y los stakeholders.

Beneficios de un alcance bien definido:

- Evita errores costosos durante el desarrollo
- Previene retrasos en la entrega del proyecto
 - Reduce costos adicionales no planificados
- Minimiza cambios innecesarios en el proceso
 - Alinea expectativas entre cliente y equipo

Funcional

a) El sistema debe permitir iniciar sesión con correo y contraseña.

→ Requisito Funcional

Describe una acción específica que el sistema debe realizar.

No Funcional

b) El sistema debe responder en menos de 2 segundos.

→ Requisito No Funcional

Define una característica de rendimiento del sistema.



Requisitos

Funcional

- c) El sistema debe permitir registrar pagos mensuales.
→ Requisito Funcional

Describe una acción específica que el sistema debe realizar.

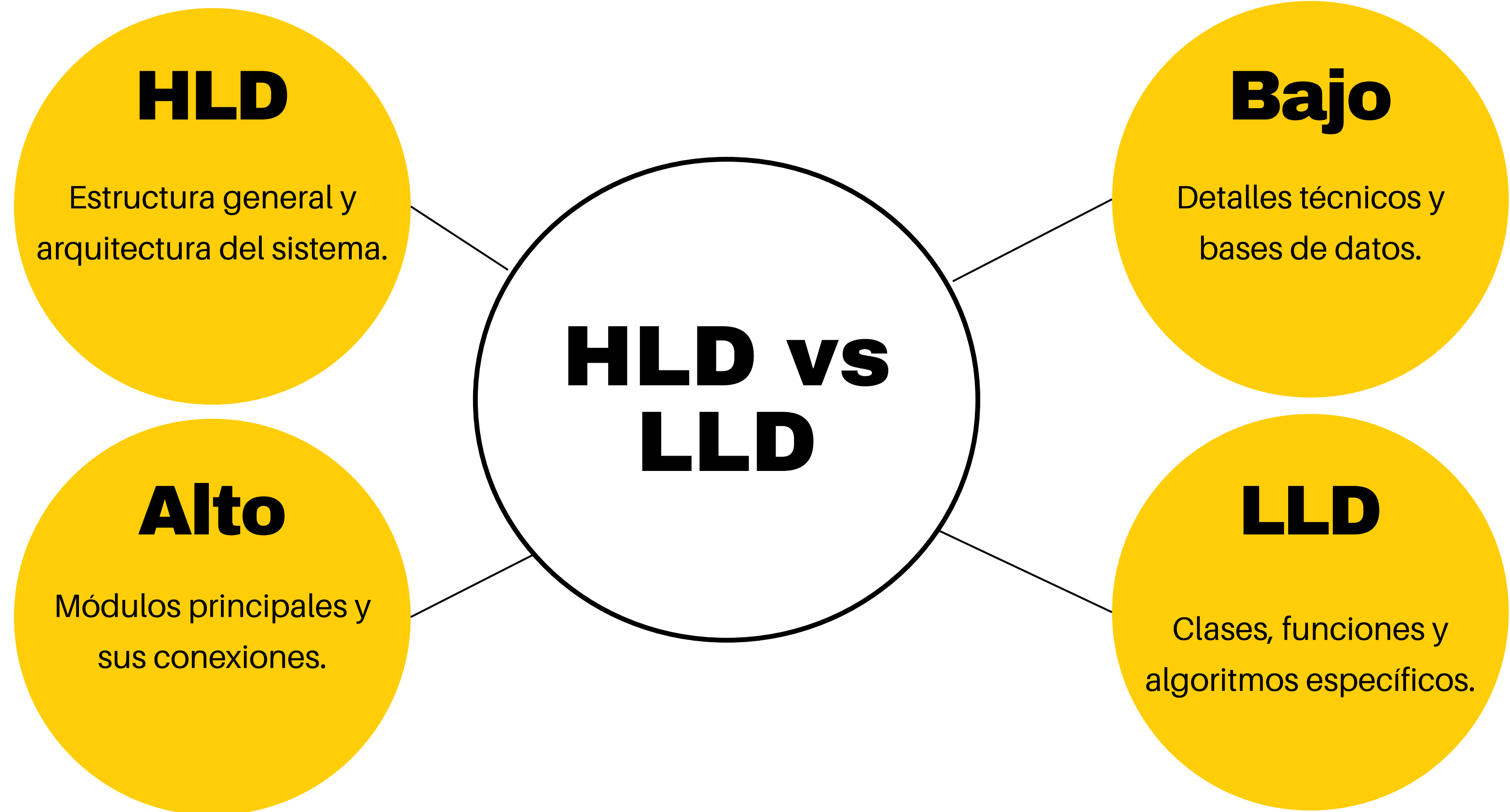
No Funcional

- d) El sistema debe soportar 500 usuarios conectados al mismo tiempo.

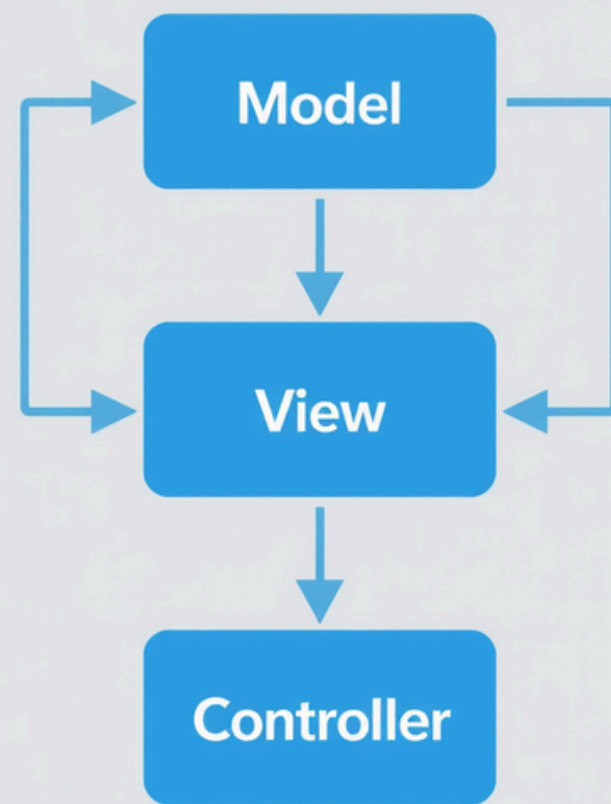
→ Requisito No Funcional

Define capacidad y rendimiento del sistema.

Parte 2



¿Cuál patrón arquitectónico separa la aplicación en Modelo, Vista y Controlador?



Arquitectura MVC

SDLC - DISEÑO DE SISTEMAS

Patrón MVC: Model-View- Controller

El patrón arquitectónico MVC separa la aplicación en tres componentes principales: el Modelo maneja los datos y la lógica de negocio, la Vista presenta la información al usuario, y el Controlador gestiona las interacciones entre ambos. Esta separación facilita el mantenimiento, la escalabilidad y las pruebas del software.

¿Qué es Clean Code?

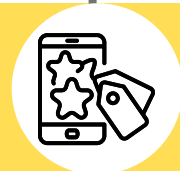
CÓDIGO LIMPIO

Clean Code es código que es:

- Claro y legible: fácil de leer y comprender a primera vista
- Ordenado y estructurado: sigue convenciones y patrones consistentes
- Fácil de mantener: permite realizar cambios sin introducir errores
- Autodocumentado: los nombres de variables y funciones explican su propósito

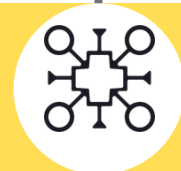
¿Qué pruebas validan funciones o métodos individuales del código?

SDLC - PREGUNTAS Y RESPUESTAS



UNITARIAS

Validan funciones o métodos individuales del código de forma aislada.



INTEGRACIÓN

Verifican la interacción correcta entre módulos o componentes.



ACEPTACIÓN

Evalúan el sistema completo en un entorno real.



SISTEMA

Confirman que el software cumple los requisitos del usuario.

¿Qué significa CI/CD?



CI: INTEGRACIÓN CONTINUA



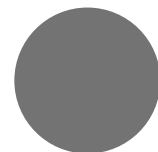
CD: ENTREGA CONTINUA



CD: DESPLIEGUE CONTINUO



AUTOMATIZACIÓN



CI/CD permite automatizar pruebas, integración y despliegue de código. Reduce errores manuales, acelera entregas y mejora la calidad del software mediante pipelines automatizados.

¿QUÉ HERRAMIENTA PERMITE EMPAQUETAR APLICACIONES EN CONTENEDORES?



SDLC - PREGUNTAS Y RESPUESTAS

Docker

¿QUÉ ES DOCKER?

Docker es la herramienta que permite empaquetar aplicaciones junto con todas sus dependencias en contenedores portátiles y ligeros.

BENEFICIOS CLAVE

Garantiza que la aplicación funcione igual en cualquier entorno. Facilita el despliegue, escalabilidad y la gestión de microservicios.

Explica la diferencia entre mantenimiento correctivo y mantenimiento evolutivo.

SDLC - CICLO DE VIDA DEL DESARROLLO DE SOFTWARE

Tipos de Mantenimiento

Correctivo

Corrige errores, bugs o fallas detectadas en el software después del despliegue. Es reactivo y responde a problemas reportados por usuarios o encontrados en producción.

Evolutivo

Agrega nuevas funcionalidades, mejoras o adaptaciones al software. Es proactivo y responde a nuevas necesidades del negocio o cambios en el entorno tecnológico.

Análisis de Caso Práctico



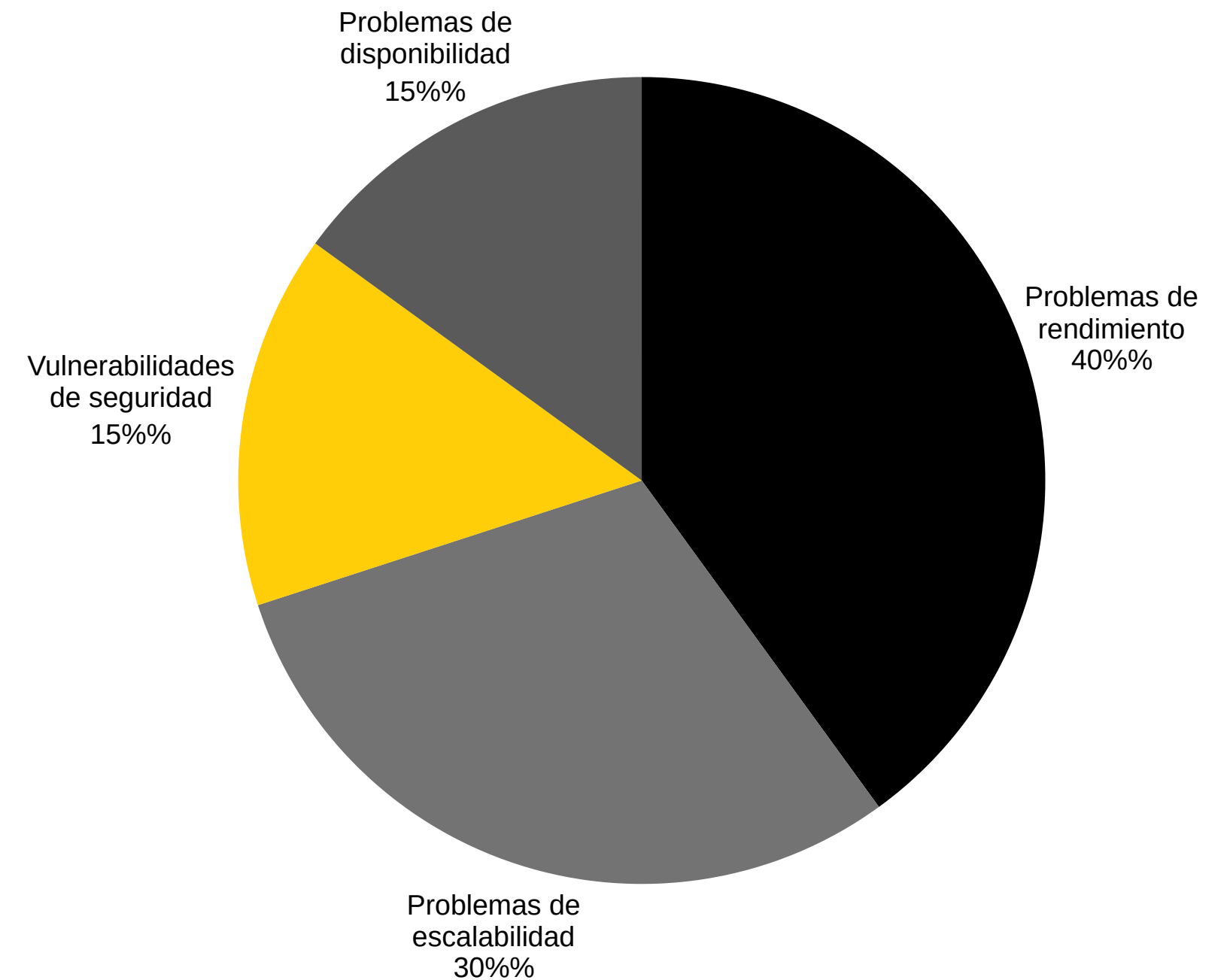
Escenario: Una aplicación bancaria experimenta lentitud severa cuando muchos usuarios se conectan simultáneamente. Los clientes reportan tiempos de espera excesivos.



Pregunta: ¿Qué tipo de requisito falló en este caso? La funcionalidad de la aplicación es correcta, pero el problema surge bajo alta demanda de usuarios.



Respuesta: Falló un requisito NO FUNCIONAL, específicamente relacionado con rendimiento y escalabilidad. El sistema no fue diseñado para soportar la carga esperada.



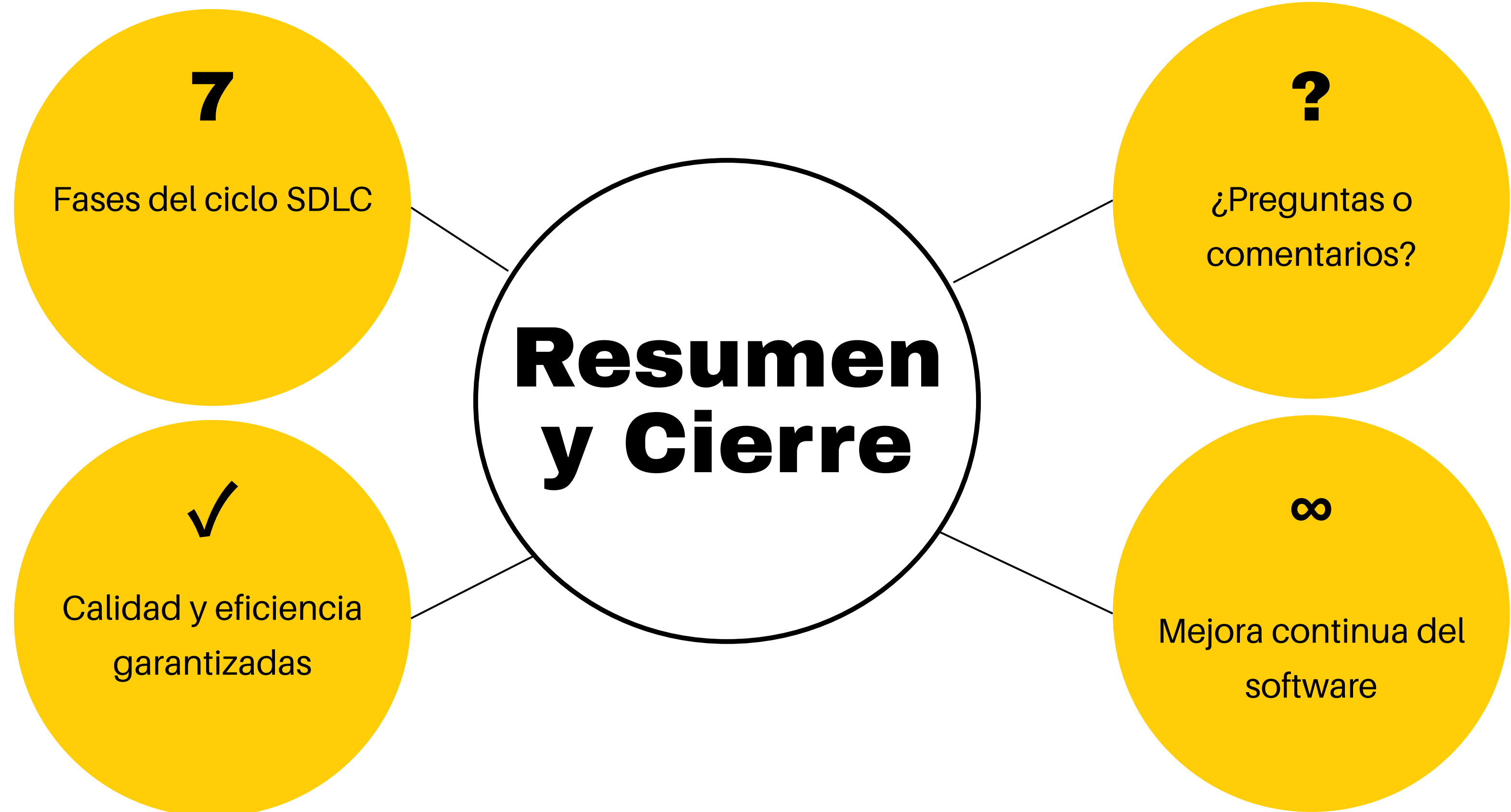
Fuente: datos generados con IA. Antes de usar el elemento, reemplaza o verifica los datos.

Detección del Problema

¿En qué fase del SDLC debió detectarse el problema de rendimiento?

Respuesta: En la fase de pruebas, específicamente durante las pruebas de carga y rendimiento. Esta fase permite identificar problemas de escalabilidad antes del despliegue.

Las pruebas de carga simulan múltiples usuarios conectados simultáneamente para verificar que el sistema cumple con los requisitos no funcionales de rendimiento. Detectar estos problemas tempranamente evita fallas críticas en producción y costos de corrección elevados.



¡Gracias por participar!